

FengHuang-OS: Open Memory Orchestration Layer for Scalable LLM Inference (Microsoft Research Asia)

Team Members

Anton Melnychuk

Email: anton.melnichuk@yale.edu

NetId: am3785

Background & Motivation

Modern Large Language Models (LLMs) are increasingly limited not by compute, but by memory. During inference, the KV cache (key/value tensors) grows linearly with sequence length and batch size, quickly exhausting GPU memory.

The 2025 paper from Microsoft *FengHuang: Towards Next-Generation AI Memory Orchestration* proposes a new AI-native memory orchestration architecture.

Rather than assuming all KV cache must live in GPU memory, FengHuang proposes intelligent placement and orchestration to maximize utilization and scalability.

The FengHuang system is presented as a vision and architecture paper:

<https://arxiv.org/html/2511.10753v1>

There is currently no polished nor MRA private validated implementation or deployable prototype. We aim to validate and advance this project to production.

Project Goal

We propose to build FengHuang-OS. Our system will:

- Implement multi-tier KV cache placement (RTL, Completed)
- Dynamically offload and prefetch KV blocks (MoE challenge, Failed)
- Support multi-GPU and multi-node inference (Hybrid Design)
- Integrate with an LLM inference backend (Stretch Goal, Missing)
- Provide prefetch simulations (Completed)
- Be fully open-sourced (Completed)
- Discuss production strategy (Completed)

Available Infrastructure (To Be Discussed)

We will deploy on the course cluster. In availability we have 8 GPU servers (H200-class GPUs), multi-GPU per node, RoCE networking, and NCCL distributed communication.

Deliverables

- Open-source GitHub repository (Completed)
- Memory orchestration runtime RTL (Completed)
- Multi-tier KV cache implementation RTL (Completed)
- Integration with LLM backend (Missing)
- Distributed cluster deployment scripts (Missing)
- Web dashboard (Missing)
- RTL Benchmark suite (Completed)
- Prefetch Strategy Simulations (Completed)
- Final report with evaluation and analysis (Completed)
- + (New) Share Results with MRA Team (Completed)

Benchmarking

- GPU memory reduction (Showed)
- CPU memory usage (Not Necessary)
- Throughput (tokens/sec) (Showed)
- Latency (TTFT, decode time) (Partial, RTL Synthesys)
- Memory transfer bandwidth (Static, Not Necessary)
- Scaling efficiency across nodes (Described, Provided Redesign)